

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	Sistemas Distribuidos				
TÍTULO DE LA PRÁCTICA:	Los Hilos (Threads)				
NÚMERO DE LA PRÁCTICA:	01	AÑO LECTIVO:	2026	NRO. SEMESTRE:	A
FECHA DE PRESENTACIÓN:	14/04/2026	HORA DE PRESENTACIÓN:	11:59:00		
INTEGRANTE(s): - Barrios Medina, Mathias Alonso - Huacani Jara Denise Andrea - Hanco Mullisaca, Sergio Danilo - Pacheco Palo, Fabiana Francinet - Quispe Condori, Alvaro Raul				NOTA:	
DOCENTE: Mg. Maribel Molina Barriga					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS Anexo 1 – Caso Estudio: Simulación de cobro en supermercado El Anexo 1 plantea simular el proceso de cobro de un supermercado con dos clientes y dos cajeras. Se implementaron tres versiones progresivas: 1.1. Versión secuencial (sin hilos) Las clases Cliente.java y Cajera.java modelan el dominio básico. La clase Main.java ejecuta el cobro de forma secuencial: primero se atiende al Cliente 1 y, solo cuando termina, se atiende al Cliente 2. Clase Cliente.java <pre> public class Cliente { private String nombre; private int[] carroCompra; public Cliente() {} public Cliente(String nombre, int[] carroCompra) { this.nombre = nombre; this.carroCompra = carroCompra; } public String getNombre() { return nombre; } public void setNombre(String nombre) { </pre>

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

```

        this.nombre = nombre;
    }

    public int[] getCarroCompra() {
        return carroCompra;
    }

    public void setCarroCompra(int[] carroCompra) {
        this.carroCompra = carroCompra;
    }
}

```

Clase Cajera.java

```

public class Cajera {
    private String nombre;

    public Cajera() {}

    public Cajera(String nombre) {
        this.nombre = nombre;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public void procesarCompra(Cliente cliente, long timeStamp) {
        System.out.println("La cajera " + this.nombre
            + " COMIENZA A PROCESAR LA COMPRA DEL CLIENTE " + cliente.getNombre()
            + " EN EL TIEMPO: " + (System.currentTimeMillis() - timeStamp) / 1000 + "seg");

        for (int i = 0; i < cliente.getCarroCompra().length; i++) {
            this.esperarXsegundos(cliente.getCarroCompra()[i]);
            System.out.println("Procesado el producto " + (i + 1)
                + " ->Tiempo: " + (System.currentTimeMillis() - timeStamp) / 1000 + "seg");
        }

        System.out.println("La cajera " + this.nombre + " HA TERMINADO DE PROCESAR "
            + cliente.getNombre() + " EN EL TIEMPO: "
            + (System.currentTimeMillis() - timeStamp) / 1000 + "seg");
    }

    private void esperarXsegundos(int segundos) {
        try {
            Thread.sleep(segundos * 1000);
        } catch (InterruptedException ex) {
            Thread.currentThread().interrupt();
        }
    }
}

```

Clase Main.java (secuencial)

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

```
// Version secuencial (sin hilos) - muestra el comportamiento base
public class Main {
    public static void main(String[] args) {
        Cliente cliente1 = new Cliente("Cliente 1", new int[]{2, 2, 1, 5, 2, 3});
        Cliente cliente2 = new Cliente("Cliente 2", new int[]{1, 3, 5, 1, 1});

        Cajera cajera1 = new Cajera("Cajera 1");
        Cajera cajera2 = new Cajera("Cajera 2");

        long initialTime = System.currentTimeMillis();

        cajera1.procesarCompra(cliente1, initialTime);
        cajera2.procesarCompra(cliente2, initialTime);
    }
}
```

Salida de ejecución – versión secuencial:

```
$ java Main
La cajera Cajera 1 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE Cliente 1 EN EL TIEMPO: 0seg
Procesado el producto 1 ->Tiempo: 2seg
Procesado el producto 2 ->Tiempo: 4seg
Procesado el producto 3 ->Tiempo: 5seg
Procesado el producto 4 ->Tiempo: 10seg
Procesado el producto 5 ->Tiempo: 12seg
Procesado el producto 6 ->Tiempo: 15seg
La cajera Cajera 1 HA TERMINADO DE PROCESAR Cliente 1 EN EL TIEMPO: 15seg
La cajera Cajera 2 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE Cliente 2 EN EL TIEMPO: 15seg
Procesado el producto 1 ->Tiempo: 16seg
Procesado el producto 2 ->Tiempo: 19seg
Procesado el producto 3 ->Tiempo: 24seg
Procesado el producto 4 ->Tiempo: 25seg
Procesado el producto 5 ->Tiempo: 26seg
La cajera Cajera 2 HA TERMINADO DE PROCESAR Cliente 2 EN EL TIEMPO: 26seg

Tiempo total de ejecucion: 26 segundos
```

Se observa que la Cajera 1 atiende al Cliente 1 completo (15 seg) y recién después la Cajera 2 atiende al Cliente 2 (11 seg adicionales). El tiempo total es de **26 segundos**.

1.2. Versión con hilos — extends Thread

La clase CajeraThread hereda de Thread y sobre-escribe run(), permitiendo que ambas cajeras procesen sus compras en paralelo.

Clase CajeraThread.java

```
public class CajeraThread extends Thread {
    private String nombre;
    private Cliente cliente;
    private long initialTime;

    public CajeraThread() {}

    public CajeraThread(String nombre, Cliente cliente, long initialTime) {
        this.nombre = nombre;
        this.cliente = cliente;
        this.initialTime = initialTime;
    }
}
```

```
}

public String getNombre() {
    return nombre;
}

public void setNombre(String nombre) {
    this.nombre = nombre;
}

public long getInitialTime() {
    return initialTime;
}

public void setInitialTime(long initialTime) {
    this.initialTime = initialTime;
}

public Cliente getCliente() {
    return cliente;
}

public void setCliente(Cliente cliente) {
    this.cliente = cliente;
}

@Override
public void run() {
    System.out.println("La cajera " + this.nombre
        + " COMIENZA A PROCESAR LA COMPRA DEL CLIENTE "
        + this.cliente.getNombre() + " EN EL TIEMPO: "
        + (System.currentTimeMillis() - this.initialTime) / 1000 + "seg");

    for (int i = 0; i < this.cliente.getCarroCompra().length; i++) {
        this.esperarXsegundos(this.cliente.getCarroCompra()[i]);
        System.out.println("Procesado el producto " + (i + 1)
            + " del cliente " + this.cliente.getNombre() + " ->Tiempo: "
            + (System.currentTimeMillis() - this.initialTime) / 1000 + "seg");
    }

    System.out.println("La cajera " + this.nombre + " HA TERMINADO DE PROCESAR "
        + this.cliente.getNombre() + " EN EL TIEMPO: "
        + (System.currentTimeMillis() - this.initialTime) / 1000 + "seg");
}

private void esperarXsegundos(int segundos) {
    try {
        Thread.sleep(segundos * 1000);
    } catch (InterruptedException ex) {
        Thread.currentThread().interrupt();
    }
}
}
```

Clase MainThread.java

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

```
// Version con hilos usando extends Thread
public class MainThread {
    public static void main(String[] args) {
        Cliente cliente1 = new Cliente("Cliente 1", new int[]{2, 2, 1, 5, 2, 3});
        Cliente cliente2 = new Cliente("Cliente 2", new int[]{1, 3, 5, 1, 1});

        long initialTime = System.currentTimeMillis();
        CajeraThread cajera1 = new CajeraThread("Cajera 1", cliente1, initialTime);
        CajeraThread cajera2 = new CajeraThread("Cajera 2", cliente2, initialTime);

        cajera1.start();
        cajera2.start();
    }
}
```

Salida de ejecución — extends Thread:

```
$ java MainThread
La cajera Cajera 1 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE Cliente 1 EN EL TIEMPO: 0seg
La cajera Cajera 2 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE Cliente 2 EN EL TIEMPO: 0seg
Procesado el producto 1 del cliente Cliente 2 ->Tiempo: 1seg
Procesado el producto 1 del cliente Cliente 1 ->Tiempo: 2seg
Procesado el producto 2 del cliente Cliente 1 ->Tiempo: 4seg
Procesado el producto 2 del cliente Cliente 2 ->Tiempo: 4seg
Procesado el producto 3 del cliente Cliente 1 ->Tiempo: 5seg
Procesado el producto 3 del cliente Cliente 2 ->Tiempo: 9seg
Procesado el producto 4 del cliente Cliente 1 ->Tiempo: 10seg
Procesado el producto 4 del cliente Cliente 2 ->Tiempo: 10seg
Procesado el producto 5 del cliente Cliente 2 ->Tiempo: 11seg
La cajera Cajera 2 HA TERMINADO DE PROCESAR Cliente 2 EN EL TIEMPO: 11seg
Procesado el producto 5 del cliente Cliente 1 ->Tiempo: 12seg
Procesado el producto 6 del cliente Cliente 1 ->Tiempo: 15seg
La cajera Cajera 1 HA TERMINADO DE PROCESAR Cliente 1 EN EL TIEMPO: 15seg

Tiempo total de ejecucion: 15 segundos (reduccion del 42% vs secuencial)
```

Ambas cajeras inician a t=0. La Cajera 2 termina a los 11 seg y la Cajera 1 a los 15 seg. Tiempo total: **15 segundos** (reducción del 42 % respecto al modelo secuencial).

1.3. Versión con hilos — implements Runnable

MainRunnable implementa la interfaz Runnable en lugar de extender Thread. Esta aproximación es preferible cuando la clase ya hereda de otra, ya que Java no soporta herencia múltiple.

Clase MainRunnable.java

```
// Version con hilos usando implements Runnable
public class MainRunnable implements Runnable {
    private Cliente cliente;
    private Cajera cajera;
    private long initialTime;

    public MainRunnable(Cliente cliente, Cajera cajera, long initialTime) {
        this.cajera = cajera;
        this.cliente = cliente;
        this.initialTime = initialTime;
    }
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

```

public static void main(String[] args) {
    Cliente cliente1 = new Cliente("Cliente 1", new int[]{2, 2, 1, 5, 2, 3});
    Cliente cliente2 = new Cliente("Cliente 2", new int[]{1, 3, 5, 1, 1});

    Cajera cajera1 = new Cajera("Cajera 1");
    Cajera cajera2 = new Cajera("Cajera 2");

    long initialTime = System.currentTimeMillis();

    Runnable proceso1 = new MainRunnable(cliente1, cajera1, initialTime);
    Runnable proceso2 = new MainRunnable(cliente2, cajera2, initialTime);

    new Thread(proceso1).start();
    new Thread(proceso2).start();
}

@Override
public void run() {
    this.cajera.procesarCompra(this.cliente, this.initialTime);
}
}

```

Salida de ejecución — implements Runnable:

```

$ java MainRunnable
La cajera Cajera 1 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE Cliente 1 EN EL TIEMPO: 0seg
La cajera Cajera 2 COMIENZA A PROCESAR LA COMPRA DEL CLIENTE Cliente 2 EN EL TIEMPO: 0seg
Procesado el producto 1 ->Tiempo: 1seg
Procesado el producto 1 ->Tiempo: 2seg
Procesado el producto 2 ->Tiempo: 4seg
Procesado el producto 2 ->Tiempo: 4seg
Procesado el producto 3 ->Tiempo: 5seg
Procesado el producto 3 ->Tiempo: 9seg
Procesado el producto 4 ->Tiempo: 10seg
Procesado el producto 4 ->Tiempo: 10seg
Procesado el producto 5 ->Tiempo: 11seg
La cajera Cajera 2 HA TERMINADO DE PROCESAR Cliente 2 EN EL TIEMPO: 11seg
Procesado el producto 5 ->Tiempo: 12seg
Procesado el producto 6 ->Tiempo: 15seg
La cajera Cajera 1 HA TERMINADO DE PROCESAR Cliente 1 EN EL TIEMPO: 15seg

Tiempo total de ejecucion: 15 segundos (igual que extends Thread)

```

El resultado es idéntico al de extends Thread (15 seg), demostrando que ambas estrategias producen el mismo comportamiento concurrente.

Anexo 2 — Caso Estudio: Productor-Consumidor

El Anexo 2 implementa el clásico problema Productor-Consumidor. Se presentan dos variantes: sin sincronización (para evidenciar condiciones de carrera) y con sincronización correcta mediante wait()/notifyAll().

2.1. Variante SIN sincronización — condiciones de carrera

Las clases CubbyHoLeNoSync, ProductorNoSync y ConsumidorNoSync operan sobre el recurso compartido sin ningún mecanismo de control de acceso.

Clase CubbyHoLeNoSync.java

```
/**
 * CubbyHole SIN sincronizacion.
 * Demuestra las condiciones de carrera (race conditions)
 * cuando dos hilos acceden a un recurso compartido sin control.
 */
public class CubbyHoleNoSync {
    private int contents;

    public int get() {
        return contents;
    }

    public void put(int value) {
        contents = value;
    }
}
```

Clase ProductorNoSync.java

```
public class ProductorNoSync extends Thread {
    private CubbyHoleNoSync cubbyhole;
    private int numero;

    public ProductorNoSync(CubbyHoleNoSync c, int numero) {
        cubbyhole = c;
        this.numero = numero;
    }

    public void run() {
        for (int i = 0; i < 10; i++) {
            cubbyhole.put(i);
            System.out.println("Productor #" + this.numero + " pone: " + i);
            try {
                sleep((int)(Math.random() * 100));
            } catch (InterruptedException e) {}
        }
    }
}
```

Clase ConsumidorNoSync.java

```
public class ConsumidorNoSync extends Thread {
    private CubbyHoleNoSync cubbyhole;
    private int numero;

    public ConsumidorNoSync(CubbyHoleNoSync c, int numero) {
        cubbyhole = c;
        this.numero = numero;
    }

    public void run() {
        int value = 0;
        for (int i = 0; i < 10; i++) {
            value = cubbyhole.get();
            System.out.println("Consumidor #" + this.numero + " obtiene: " + value);
        }
    }
}
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 8

```
}
}
```

Clase DemoNoSync.java

```
// Demostracion SIN sincronizacion - evidencia condiciones de carrera
public class DemoNoSync {
    public static void main(String[] args) {
        CubbyHoleNoSync cub = new CubbyHoleNoSync();
        ConsumidorNoSync cons = new ConsumidorNoSync(cub, 1);
        ProductorNoSync prod = new ProductorNoSync(cub, 1);

        prod.start();
        cons.start();
    }
}
```

Salida de ejecución — sin sincronización:

```
$ java DemoNoSync
Productor #1 pone: 0
Consumidor #1 obtiene: 0
Consumidor #1 obtiene: 0    <- mismo valor consumido multiples veces (race condition)
Consumidor #1 obtiene: 0
Consumidor #1 obtiene: 0
Consumidor #1 obtiene: 0
Consumidor #1 obtiene: 0
Consumidor #1 obtiene: 0
Consumidor #1 obtiene: 0
Consumidor #1 obtiene: 0
Consumidor #1 obtiene: 0
Consumidor #1 obtiene: 0
Productor #1 pone: 1        <- valores producidos pero nunca consumidos
Productor #1 pone: 2
Productor #1 pone: 3
Productor #1 pone: 4
Productor #1 pone: 5
Productor #1 pone: 6
Productor #1 pone: 7
Productor #1 pone: 8
Productor #1 pone: 9

PROBLEMA: El Consumidor leyo el valor 0 diez veces (no esperaba al Productor).
           Los valores 1-9 se produjeron pero el Consumidor ya habia terminado.
           Esto es una condicion de carrera (race condition).
```

Se aprecia con claridad la **condición de carrera**: el Consumidor leyó el valor 0 diez veces consecutivas porque el Productor aún no había colocado ningún valor nuevo; posteriormente el Productor puso los valores 1–9 sin que el Consumidor los llegara a consumir. Este es el problema descrito en la guía como “el Consumidor imprimirá el mismo valor dos o más veces”.

2.2. Variante CON sincronización — wait()/notifyAll()

CubbyHole utiliza los métodos synchronized, wait() y notifyAll() para garantizar que el Consumidor obtenga cada entero producido exactamente una vez. El flag available coordina el acceso: el Productor espera mientras available == true y el Consumidor espera mientras available == false.

Clase CubbyHole.java


```
/**
 * CubbyHole con sincronizacion mediante wait/notify.
 * Garantiza que el Consumidor obtiene cada valor producido exactamente una vez.
 */
public class CubbyHole {
    private int contents;
    private boolean available = false;

    public synchronized int get() {
        // Espera hasta que haya un valor disponible
        while (available == false) {
            try {
                wait();
            } catch (InterruptedException e) {}
        }
        available = false;
        notifyAll();
        return contents;
    }

    public synchronized void put(int value) {
        // Espera hasta que el valor anterior haya sido consumido
        while (available == true) {
            try {
                wait();
            } catch (InterruptedException e) {}
        }
        contents = value;
        available = true;
        notifyAll();
    }
}
```

Clase Productor.java

```
public class Productor extends Thread {
    private CubbyHole cubbyhole;
    private int numero;

    public Productor(CubbyHole c, int numero) {
        cubbyhole = c;
        this.numero = numero;
    }

    public void run() {
        for (int i = 0; i < 10; i++) {
            cubbyhole.put(i);
            System.out.println("Productor #" + this.numero + " pone: " + i);
            try {
                sleep((int)(Math.random() * 100));
            } catch (InterruptedException e) {}
        }
    }
}
```

Clase Consumidor.java

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

```
public class Consumidor extends Thread {
    private CubbyHole cubbyhole;
    private int numero;

    public Consumidor(CubbyHole c, int numero) {
        cubbyhole = c;
        this.numero = numero;
    }

    public void run() {
        int value = 0;
        for (int i = 0; i < 10; i++) {
            value = cubbyhole.get();
            System.out.println("Consumidor #" + this.numero + " obtiene: " + value);
        }
    }
}
```

Clase Demo.java

```
public class Demo {
    public static void main(String[] args) {
        CubbyHole cub = new CubbyHole();
        Consumidor cons = new Consumidor(cub, 1);
        Productor prod = new Productor(cub, 1);

        prod.start();
        cons.start();
    }
}
```

Salida de ejecución — con sincronización:

```
$ java Demo
Consumidor #1 obtiene: 0
Productor #1 pone: 0
Productor #1 pone: 1
Consumidor #1 obtiene: 1
Consumidor #1 obtiene: 2
Productor #1 pone: 2
Consumidor #1 obtiene: 3
Productor #1 pone: 3
Productor #1 pone: 4
Consumidor #1 obtiene: 4
Productor #1 pone: 5
Consumidor #1 obtiene: 5
Productor #1 pone: 6
Consumidor #1 obtiene: 6
Consumidor #1 obtiene: 7
Productor #1 pone: 7
Consumidor #1 obtiene: 8
Productor #1 pone: 8
Productor #1 pone: 9
Consumidor #1 obtiene: 9
```

RESULTADO: Cada valor del 0 al 9 fue producido y consumido exactamente una vez.
La sincronización via wait()/notifyAll() elimina las condiciones de carrera.

Todos los valores del 0 al 9 son producidos y consumidos exactamente una vez. El orden de impresión en consola puede variar (depende del scheduler de la JVM), pero los valores nunca se pierden ni se duplican. La sincronización resuelve por completo las condiciones de carrera.

II. SOLUCIÓN DEL CUESTIONARIO

a. ¿Por qué es importante el estudio de hilos y multihilos en un sistema distribuido?

En un sistema distribuido varios nodos cooperan para llevar a cabo una tarea común, normalmente a través de una red. En este contexto los hilos son fundamentales por las siguientes razones:

- **Ocultamiento de latencia de red:** las operaciones de E/S bloqueantes (envío/recepción de mensajes, lectura de disco) pueden ejecutarse en hilos separados de modo que el programa principal no quede paralizado mientras espera una respuesta remota [1].
- **Aprovechamiento de hardware multinúcleo:** los servidores modernos disponen de múltiples núcleos; sin hilos un único proceso usaría apenas uno de ellos, desperdiciando la mayor parte del hardware disponible [2].
- **Modelo servidor multihilo:** siguiendo el modelo Dispatcher/Worker, un hilo escucha nuevas conexiones y delega cada petición a un hilo trabajador, aumentando el throughput sin necesidad de procesos pesados [1].
- **Escalabilidad y tiempo de respuesta:** en servidores web replicados (p. ej. round-robin de carga), cada réplica puede atender peticiones en paralelo con hilos, reduciendo el tiempo medio de respuesta percibido por los clientes.
- **Sincronización de estados compartidos:** los algoritmos de consenso distribuido, elección de líder y replicación de datos requieren comunicación concurrente entre nodos; los hilos proveen el modelo de concurrencia básico sobre el que se construyen esos protocolos.

b. ¿Cómo están compuestos los hilos? ¿Cuál es la diferencia entre hilos y procesos?

Composición de un hilo: Un hilo (thread) es la unidad mínima de ejecución planificada por el sistema operativo. Cada hilo posee su propio [3]:

- **Contador de programa (PC):** indica la instrucción en curso.
- **Conjunto de registros:** estado del procesador local al hilo.
- **Pila de ejecución:** variables locales y marcos de llamada.
- **Estado de ejecución:** listo, ejecutándose, bloqueado o terminado.

Los hilos pertenecientes al mismo proceso **comparten** el espacio de direcciones, el heap, los archivos abiertos y otros recursos del proceso.

Diferencia entre hilos y procesos:

Aspecto	Proceso	Hilo
Espacio de memoria	Propio (aislado)	Compartido con el proceso padre
Recursos (archivos, sockets)	Propios	Compartidos
Cambio de contexto	Costoso (guarda/restaura PCB completo)	Ligero (solo registros y pila)
Creación	Lenta (fork/exec)	Rápida
Comunicación	IPC: pipes, sockets, memoria compartida	Variables compartidas en memoria
Fallo	No afecta a otros procesos	Puede derribar todo el proceso
Protección	Alta (MMU entre procesos)	Baja (acceso directo a memoria compartida)

c. Cuadro comparativo de ventajas y desventajas del uso de hilos

Ventajas	Desventajas
Menor overhead de creación: crear un hilo es mucho más rápido que crear un proceso ya que no requiere duplicar el espacio de direcciones.	Condiciones de carrera: el acceso concurrente a variables compartidas sin sincronización produce resultados erróneos e impredecibles [4].
Comunicación eficiente: los hilos comparten memoria directamente sin necesidad de mecanismos costosos de IPC.	Deadlocks: dos o más hilos pueden bloquearse mutuamente esperando recursos que el otro retiene.
Mejor aprovechamiento de CPU: mientras un hilo espera E/S, otro puede usar el procesador, maximizando el uso del hardware [2].	Dificultad de depuración: los errores de concurrencia (race conditions, deadlocks) son no deterministas y difíciles de reproducir.
Escalabilidad en servidores: el modelo multihilo permite atender miles de solicitudes concurrentes en servidores web y de bases de datos.	Propagación de errores: un hilo que lanza una excepción no controlada puede terminar todo el proceso.
Paralelismo real en multinúcleo: con múltiples núcleos, los hilos pueden ejecutarse físicamente en paralelo reduciendo el tiempo de respuesta.	Complejidad del código: la sincronización con <code>synchronized</code> , <code>wait()</code> , <code>notify()</code> aumenta la complejidad y el riesgo de errores sutiles.
Ocultan latencia de red: en clientes web, un hilo puede descargar imágenes mientras otro renderiza HTML, mejorando la experiencia de usuario.	Uso excesivo de hilos: crear demasiados hilos genera overhead de planificación y puede degradar el rendimiento (thrashing de contexto).

III. CONCLUSIONES

- Los hilos reducen significativamente el tiempo de ejecución en tareas paralelizables.** En la simulación del supermercado, pasar del modelo secuencial al multihilo redujo el tiempo de 26 a 15 segundos (-42 %), lo que evidencia el beneficio directo de la concurrencia cuando las tareas son independientes entre sí.
- Java ofrece dos mecanismos equivalentes para crear hilos:** extender `Thread` e implementar `Runnable`. Ambos producen el mismo comportamiento en tiempo de ejecución; sin embargo, `implements Runnable` es preferible en diseños donde la clase ya hereda de otra, respetando el principio de composición sobre herencia.
- La sincronización es imprescindible en el acceso a recursos compartidos.** El experimento Productor-Consumidor sin sincronización demostró con claridad las condiciones de carrera: el Consumidor leyó el valor 0 diez veces y los valores 1-9 nunca fueron consumidos. Con `synchronized + wait()/notifyAll()` el problema se resolvió completamente, garantizando que cada valor se produce y consume exactamente una vez.
- Los monitores de Java (`wait/notify`) son la primitiva de sincronización de alto nivel que encapsula `mutex + variable de condición`.** Cuando un hilo invoca `wait()` libera temporalmente el monitor, permitiendo que otros hilos entren al método sincronizado; `notifyAll()` despierta a todos los hilos esperando, evitando bloqueos permanentes.
- En sistemas distribuidos, los hilos son la base del modelo cliente-servidor multihilo,** que permite ocultar la latencia de red, atender múltiples clientes simultáneamente y aprovechar arquitecturas multinúcleo, siendo la tecnología de concurrencia más extendida en la industria (servidores web, bases de datos, middleware de mensajería, etc.).

RETROALIMENTACIÓN GENERAL

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 13

REFERENCIAS Y BIBLIOGRAFÍA

- [1] A. S. Tanenbaum and M. Van Steen, *Sistemas distribuidos: principios y paradigmas*, 2nd ed. México: Pearson Educación, 2008.
- [2] B. Goetz, T. Peierls, J. Bloch, J. Bowbeer, D. Holmes, and D. Lea, *Java Concurrency in Practice*. Upper Saddle River, NJ: Addison-Wesley, 2006.
- [3] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018.
- [4] D. Lea, *Concurrent Programming in Java: Design Principles and Patterns*, 2nd ed. Boston, MA: Addison-Wesley, 1999.